

NAME: SAMRUDDHI SUNIL BATE

ROLL NO: 14

PRN NO: 12520064

BATCH: 01

CLASS: SY-CS-SEDA-B

SUBJECT: OS

Operating System Simulation (Phase-1)

○ Aim :

To simulate a simple Operating System that can execute basic instructions.

○ Objective :

- To understand how an Operating System works
- To learn how instructions are executed
- To perform input and output operations
- To understand memory and register usage

○ Theory :

- An Operating System (OS) is system software that manages computer hardware and software resources.
- In this experiment, a simple OS is simulated with:
 - Memory represented by a 2D array M[100][4]
 - Instruction Register (IR) to hold current instruction
 - General Register (R) for temporary storage
 - Instruction Counter (IC) to track execution

○ **Control Cards :**

Control Card	Description
\$AMJ	Start of job
\$DTA	Start of data
\$END	End of job

○ **Instructions Used:**

Instruction	Full Form	Function
GD	Get Data	Reads data from input file to memory
PD	Put Data	Prints data from memory to output file
LR	Load Register	Loads data from memory to register
SR	Store Register	Stores data from register to memory
H	Halt	Stops execution

○ **Algorithm:**

Step No.	Description
1	Start the program
2	Initialize memory with blank spaces
3	Read input file line by line
4	If \$AMJ found → initialize memory
5	Load instructions into memory
6	If \$DTA found → start execution
7	Fetch instruction using IC
8	Decode and execute instruction
9	Repeat until H instruction
10	Stop the program

○ Program:

```
#include <iostream>

#include <fstream>

using namespace std;


char M[100][4]; // Memory
char IR[4];     // Instruction
char R[4];      // Register
int IC = 0;     // Counter


ifstream fin("input.txt");
ofstream fout("output.txt");


char buffer[40];


// Initialize memory
void init()
{
    for(int i=0;i<100;i++)
        for(int j=0;j<4;j++)
            M[i][j] = ' ';
}


// Execute program
void execute()
{
    while(true)
    {
        // Load instruction
```

```

for(int i=0;i<4;i++)
    IR[i] = M[IC][i];

IC++;

// GD (Read)
if(IR[0]=='G' && IR[1]=='D')
{
    int loc = (IR[2]-'0')*10 + (IR[3]-'0');
    fin.getline(buffer,40);

    int k=0;
    for(int i=0;i<10;i++)
        for(int j=0;j<4;j++)
            M[loc+i][j] = (buffer[k] != '\0') ? buffer[k++] : ' ';
}

// PD (Print)
else if(IR[0]=='P' && IR[1]=='D')
{
    int loc = (IR[2]-'0')*10 + (IR[3]-'0');

    for(int i=0;i<10;i++)
        for(int j=0;j<4;j++)
            fout << M[loc+i][j];

    fout << endl;
}

// LR (Load Register)
else if(IR[0]=='L' && IR[1]=='R')

```

```

    {
        int loc = (IR[2]-'0')*10 + (IR[3]-'0');
        for(int i=0;i<4;i++)
            R[i] = M[loc][i];
    }

    // SR (Store Register)
    else if(IR[0]=='S' && IR[1]=='R')
    {
        int loc = (IR[2]-'0')*10 + (IR[3]-'0');
        for(int i=0;i<4;i++)
            M[loc][i] = R[i];
    }

    // H (Stop)
    else if(IR[0]=='H')
    {
        break;
    }
}

// Load program
void load()
{
    int m = 0;

    while(fin.getline(buffer,40))
    {
        if(buffer[0] == '$' && buffer[1] == 'A') // $AMJ
        {

```

```

        init();

        IC = 0;

        m = 0;
    }
    else if(buffer[0] == '$' && buffer[1] == 'D') // $DTA
    {
        execute();
    }
    else if(buffer[0] == '$' && buffer[1] == 'E') // $END
    {
        continue;
    }
    else
    {
        int k=0;
        for(int j=0;j<4;j++)
            M[m][j] = buffer[k++];

        m++;
    }
}

// Main
int main()
{
    load();

    fin.close();

    fout.close();

    return 0;
}

```

```
≡ input.txt ×  
≡ input.txt  
1  $AMJ  
2  GD10  
3  PD10  
4  H  
5  $DTA  
6  HELLO SAMRUDDHI  
7  $END  
8  
9  $AMJ  
10 GD20  
11 LR20  
12 SR30  
13 PD30  
14 H  
15 $DTA  
16 OPERATING SYSTEM LAB  
17 $END //register load and store
```

```
≡ output.txt ×  ≡ phase1.exe  
≡ output.txt  
1  HELLO SAMRUDDHI  
2  OPER  
3
```

○ Output Explanation:

Job Instruction Explanation

1	GD10	Reads data into memory
	PD10	Prints data
	H	Stops execution
2	GD20	Reads data
	LR20	Loads into register
	SR30	Stores into new memory location
	PD30	Prints data

○ Limitations :

No.	Limitation
1	No error handling
2	No interrupt mechanism
3	No memory protection
4	Limited instruction set

○ Conclusion :

The experiment successfully demonstrates the basic working of an Operating System, including instruction execution and memory handling.